

# De onde vêm os dados

Encontro 2 — HTTP, JSON e persistência sem ORM

---

**2h**

de aula

**24min**

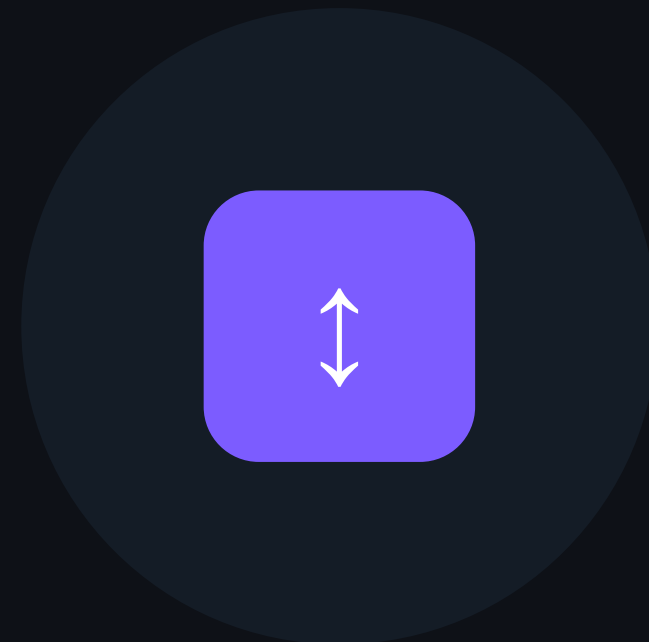
de exposição

**96min**

de código

**5**

status codes



# Como as próximas duas horas funcionam

Mesma regra: exposição em blocos de 8 minutos, o resto é vocês programando.

MIN 00–36

## Bloco 1 — HTTP

Anatomia da requisição, arqueologia do Network e o primeiro servidor Express.

MIN 36–74

## Bloco 2 — JSON

Verbos, status, validação de entrada e a rota POST.

MIN 74–112

## Bloco 3 — Banco

SQL parametrizado, SQLite e a troca do arquivo falso pela API real.

MIN 112–120

## Fechamento

Briefing da atividade e política de IA no backend.

Hoje o momento mais importante da semana acontece aos 108 minutos: vamos trocar UMA linha no frontend e ele vai passar a falar com um banco de dados de verdade. Prestem atenção quando chegarmos lá.

# Do encontro passado. Sem consultar nada.

1 Qual arquivo do frontend pode chamar ``fetch``? E por quê?

2 Por que a lista filtrada NÃO fica guardada no estado?

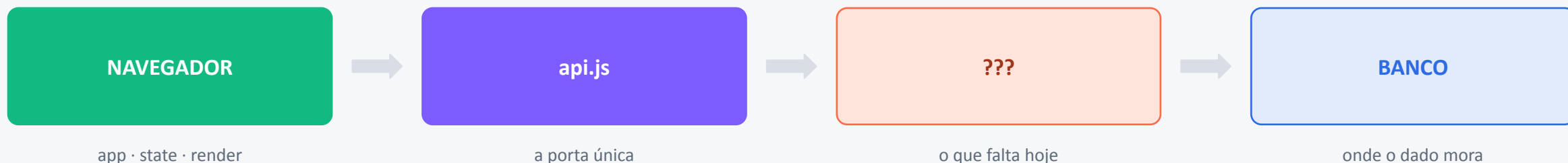
3 ``fetch`` lança exceção quando o servidor responde 404?

4 ``min-width`` acrescenta ou desfaz regra de CSS?

5 Complete: “O estado muda, a tela é \_\_\_\_\_”.

Se você errou a 3ª, ela vai te custar caro hoje: metade dos bugs de integração nasce aí.

# O mapa do sistema — e o buraco no meio dele



## O QUE JÁ FUNCIONA

### Tudo à esquerda

O frontend inteiro está pronto e não vai mudar hoje — exceto por uma linha. Ele já sabe pedir, tratar erro e desenhar.

## O QUE FALTA

### O servidor

Alguém que receba o pedido, decida o que fazer, converse com o banco e responda em JSON. É o que vocês vão escrever hoje.

BLOCO 01

# O que é HTTP, afinal

Um protocolo de conversa. Uma pergunta, uma resposta, e um esquecimento imediato.



# O balcão de atendimento

## A REQUISIÇÃO

### Você chega e faz um pedido

Você diz o que quer fazer (“quero consultar”), diz sobre o quê (“o prontuário 42”), mostra documentos se precisar (cabeçalhos) e entrega um formulário preenchido se for um cadastro (corpo).

Sem essas quatro coisas, o atendente não tem como te ajudar.

## A RESPOSTA

### O atendente responde

Primeiro ele te dá um veredito curto (“achei”, “não achei”, “seu formulário está errado”, “o sistema caiu”) — isso é o status.

Depois entrega o conteúdo, se houver, com uma etiqueta dizendo em que formato ele está.

A analogia tem um limite importante, e é o próximo slide: esse atendente esquece você completamente assim que vira as costas.

# Quatro partes. Sempre as mesmas quatro.

## O QUE SAI DO SEU NAVEGADOR

```
POST /api/patients HTTP/1.1      <- metodo + caminho
Host: localhost:3000
Content-Type: application/json    <- cabecalhos
Accept: application/json

{                                <- corpo
  "name": "Joana Ribeiro",
  "birthDate": "1990-04-12",
  "nationalId": "700012345678999"
}
```

### MÉTODO

O que eu quero fazer · GET, POST, PUT, DELETE

### CAMINHO

Sobre qual recurso · /api/patients/42

### CABEÇALHOS

Informações sobre o pedido · Content-Type, Accept

### CORPO

O dado em si — só quando há · opcional em GET

# Três partes — e a primeira é a mais importante

## O QUE VOLTA DO SERVIDOR

```
HTTP/1.1 201 Created      <- status
Content-Type: application/json <- cabecalhos
Content-Length: 118

{                          <- corpo
  "id": 9,
  "name": "Joana Ribeiro",
  "birthDate": "1990-04-12",
  "active": true
}
```

## STATUS

### O veredito, em número

É a única parte que TODO cliente entende sem combinar nada antes. Programa automático lê o status — não o texto do corpo.

## CABEÇALHOS

Metadados: formato, tamanho, cache, autenticação.

## CORPO

O conteúdo. Pode não existir — e em alguns status não DEVE existir.

# O atendente com amnésia total

Cada requisição chega como se fosse a primeira da história. O servidor não lembra de você.

## O QUE ISSO SIGNIFICA

### Nada persiste entre chamadas

- A requisição precisa carregar tudo o que o servidor precisa saber
- Quem é você vai em cabeçalho, toda vez
- O servidor não guarda “onde você parou”

## POR QUE ISSO É BOM

### É o que permite escalar

- Qualquer servidor atende qualquer requisição
- Caiu um? O próximo assume sem perceber
- Dobrar a capacidade é subir mais uma cópia

A armadilha prática: guardar dado do usuário numa variável do seu servidor (`let usuarioLogado = ...`) funciona perfeitamente na sua máquina. E quebra no dia em que a aplicação subir com duas instâncias — porque metade das requisições vai cair na cópia que não sabe de nada.

# A semântica é contrato, não decoração

GET

## Ler

Não muda nada no servidor. Pode ser repetido à vontade, pode ser cacheado, pode ser refeito pelo navegador sozinho.

POST

## Criar

Cria algo novo. Repetir cria de novo — dois cliques, dois pacientes.

PUT / PATCH

## Alterar

PUT substitui o recurso inteiro. PATCH altera só os campos enviados.

DELETE

## Remover

Remove. Repetir não causa dano adicional: já estava removido.

**Bug conceitual grave:** uma rota ``GET /api/patients/42/delete``. O navegador pode disparar GET sozinho — em pré-carregamento, em cache, num crawler. Um dia você perde dados sem ninguém ter clicado em nada.

# Fazer duas vezes dá o mesmo resultado que fazer uma vez?

Palavra difícil, ideia simples — e ela decide o que acontece quando a internet do usuário oscila.

## IDEMPOTENTE

### GET • PUT • DELETE

Apertar o botão do elevador dez vezes. O elevador vem uma vez.

Se a resposta se perdeu no caminho, o cliente pode simplesmente tentar de novo, sem medo.

## NÃO IDEMPOTENTE

### POST

Apertar duas vezes o botão de comprar. Dois pedidos.

Duplo clique no “Cadastrar” cria dois pacientes com o mesmo CNS. É por isso que hoje vocês vão desabilitar o botão durante o envio.

Existem soluções sérias para isso — chave de idempotência, token de requisição. Não hoje. Hoje basta você saber que o problema existe e ter o nome dele.

# A primeira coisa que um programa lê da sua resposta

**2xx**

**Deu certo**

O pedido foi entendido e atendido.

**3xx**

**Vá para outro lugar**

Redirecionamento. Não usaremos hoje.

**4xx**

**A culpa é do pedido**

O cliente errou. Se ele não mudar nada, vai errar de novo.

**5xx**

**A culpa é minha**

O servidor quebrou. O cliente pode tentar de novo mais tarde.

A pergunta que resolve 90% dos casos: de quem é a culpa? Se o cliente precisa corrigir algo, é 4xx. Se ele não pode fazer nada, é 5xx.

## Não precisa decorar os 60. Precisa dominar estes.

200

**OK**

Li e devolvi o que você pediu.

*GET que encontrou*

201

**Created**

Criei. E devolvo o recurso criado, com o id que gerei.

*POST bem-sucedido*

400

**Bad Request**

Seu pedido está malformatado ou incompleto. Corrija e tente de novo.

*validação falhou*

404

**Not Found**

Esse recurso não existe. Não é erro meu, não é erro seu: só não existe.

*GET /patients/9999*

409

**Conflict**

Seu pedido faz sentido, mas conflita com o estado atual.

*CNS já cadastrado*

201 devolve o recurso criado porque o cliente não tem como adivinhar o `id` que o banco gerou. Devolver poupa uma requisição inteira.

# 200 com o erro escondido no corpo

## O QUE MUITA API BRASILEIRA FAZ

```
HTTP/1.1 200 OK

{
  "sucesso": false,
  "mensagem": "name e obrigatorio"
}
```

## O QUE O PROTOCOLO PEDE

```
HTTP/1.1 400 Bad Request

{
  "error": "O campo 'name' e obrigatorio."
}
```

## POR QUE ISSO CUSTA CARO

### Porque ninguém lê o seu corpo

- O monitoramento conta erros por faixa de status: com 200, seu sistema parece perfeito enquanto está falhando
- O `response.ok` do fetch vira `true` — todo cliente precisa de código especial para a sua API
- Proxy, cache e balanceador tomam decisões pelo status. Você mentiu para todos eles
- Quem for integrar com você vai precisar ler a documentação para descobrir o óbvio

# Arqueologia do Network

Abra o DevTools (F12) na aba Network, marque “Preserve log” e recarregue a página.

1

## Abra qualquer site que você use

Portal do IFPI, um e-commerce, uma rede social. Recarregue.

2

## Ache UMA requisição de documento

Tipo `document`. Anote: método, status, Content-Type.

3

## Ache UMA requisição de API

Filtre por `Fetch/XHR`. Abra a aba Response: é JSON?

4

## Ache um status que não seja 200

Quase todo site tem um 304, um 204 ou um 404. Anote qual e o que ele significa.

### PREENCHA A FICHA

#### O que anotar

- Método e caminho
- Status e o que ele significa
- Content-Type da resposta
- Tamanho e tempo

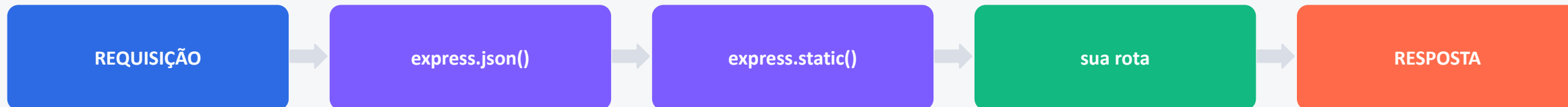
### O PONTO

#### Nada disso é abstrato

Tudo que você viu nos últimos slides está acontecendo agora, no seu navegador, dezenas de vezes por segundo.

# Middleware: a esteira de triagem

Antes de chegar na rota, a requisição passa por uma fila de funções. Cada uma pode ler, alterar ou barrar.



`EXPRESS.JSON()`

## Traduz o corpo

Se o Content-Type for ``application/json``, ele lê o texto que chegou, converte em objeto JavaScript e coloca em ``req.body``.

Sem ele, ``req.body`` é ``undefined``. Guarde isso.

`EXPRESS.STATIC('PUBLIC')`

## Entrega os arquivos

Qualquer pedido que não bata com uma rota é procurado dentro de ``public/``.

É por isso que o frontend e a API vivem no MESMO endereço — e é por isso que não precisamos falar de CORS ainda.

**A ordem importa: middleware declarado depois da rota não roda para ela.**

# O servidor inteiro em 14 linhas

## LINHA A LINHA

```
import express from 'express';

const app = express();
const PORT = 3000;

app.use(express.json()); // lê corpo JSON
app.use(express.static('public')); // serve o frontend

app.get('/api/health', (_request, response) => {
  response.json({ status: 'ok' });
});

app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`);
});
```

## ROTA DE SAÚDE

### Por que ela existe

Serve para responder “o servidor está de pé?” sem tocar no banco. Todo serviço em produção tem uma.

``_REQUEST``

### O underline não é enfeite

Ele diz ao TypeScript: “sei que não uso este parâmetro, é de propósito”.

## SEM CAMADAS

Um arquivo só, de propósito. Arquitetura é assunto da Semana 3.

# Sua primeira rota

TODO 1 em `src/server.ts`. Ainda SEM banco: devolva um array escrito na mão.

1

## Rode ``npm run dev``

Confira ``/api/health`` no navegador antes de escrever qualquer coisa.

2

## Crie ``GET /api/patients``

Devolva um array com 2 pacientes escritos à mão, em camelCase.

3

## Abra no navegador

``localhost:3000/api/patients``. Você deve ver JSON cru na tela.

4

## Abra o Network e olhe

Qual o status? Qual o Content-Type? Quem definiu isso — você ou o Express?

### SE TRAVAR

#### Perguntas que destravam

- O terminal mostra erro de TypeScript?
- O ``tsx watch`` reiniciou depois de salvar?
- O caminho tem ``/api`` no começo?
- Você usou ``response.json(...)`` e não ``return``?

### NÍVEL 3

#### Para quem terminou

- Faça ``/api/patients?active=true`` filtrar o array
- Descubra: query string chega em ``req.query``

BLOCO 02

# JSON e entrada de dados

Receber é mais perigoso que devolver. Tudo que entra é suspeito.



# Não é um objeto JavaScript. É texto.

Pela rede não trafega objeto. Trafega uma sequência de caracteres — e alguém precisa traduzir nas duas pontas.

## ANALOGIA

### A carta e a pessoa

Você não envia uma pessoa pelo correio. Envia uma carta que descreve o que ela disse.

JSON é a carta. O objeto é a pessoa.

## O CICLO COMPLETO

```
// NAVEGADOR - objeto vira texto
const patient = { name: 'Joana' };
const texto = JSON.stringify(patient);
// '{"name":"Joana"}' <- isto viaja pela rede

// SERVIDOR - texto vira objeto de novo
app.use(express.json()); // faz o JSON.parse por voce
// req.body -> { name: 'Joana' }
```

**JSON só tem seis tipos: texto, número, booleano, nulo, objeto e lista.**

**Não tem Date, não tem função, não tem undefined. É por isso que datas viajam como texto ("1990-04-12") — e por isso a validação de formato de data é sua responsabilidade.**

# `req.body` é `undefined`

Você vai passar por isso hoje. Vai perder 10 minutos. Este slide existe para reduzir para 10 segundos.

## CAUSA 1

### Faltou `express.json()`

Sem esse middleware, ninguém lê o corpo. O Express não faz isso sozinho — é opt-in de propósito.

Confira se ele está ANTES das rotas.

## CAUSA 2

### Faltou o Content-Type

O cliente mandou o corpo mas não disse que era JSON.

O `express.json()` só age quando o cabeçalho diz `application/json`. Sem ele, ignora em silêncio.

## COMO DESCOBRIR EM 10S

### Um `console.log`

```
console.log(request.headers['content-type'],  
request.body)
```

Se o header vier vazio, o problema está no cliente.  
Se vier certo e o body vazio, no middleware.

O caso 9 do `requests.http` é exatamente esse erro, de propósito. Rode e veja acontecer antes de acontecer com você por acidente.

# Três lugares, três propósitos diferentes

## Path param

`/api/patients/42`

IDENTIFICA um recurso específico. Faz parte do endereço: sem ele, o endereço é outro.

`req.params.id`

## Query string

`/api/patients?active=true`

FILTRA, ordena, pagina. Modifica a consulta, não muda qual recurso é.

`req.query.active`

## Corpo

`POST /api/patients`

TRANSPORTA o dado novo. Só existe em POST, PUT e PATCH.

`req.body.name`

**Erro comum: mandar dado de cadastro por query string. Fica no histórico do navegador, no log do servidor e no Referer. Nunca faça isso com dado de paciente.**

# Rejeitar cedo, com mensagem que ensina

SRC/SERVER.TS

```
const ISO_DATE = /^\\d{4}-\\d{2}-\\d{2}$/;

function isBlank(value: unknown): boolean {
  return typeof value !== 'string' || value.trim() === '';
}

function validatePatientInput(body: any): string | null {
  if (isBlank(body?.name)) {
    return "O campo 'name' é obrigatorio.";
  }
  if (isBlank(body?.birthDate) || !ISO_DATE.test(body.birthDate)) {
    return "O campo 'birthDate' deve ser AAAA-MM-DD.";
  }
  return null; // null = esta tudo certo
}
```

POR QUE `ISBLANK`

## String vazia não é ausência

"" "" passa no teste de “existe”. E é lixo. Um nome de paciente com três espaços não serve para ninguém.

POR QUE DEVOLVER TEXTO

## `null` ou a mensagem

Uma função, uma resposta. Quem chama só pergunta: “tem problema?”.

E O FRONT?

Validar no frontend é conveniência. Validar no backend é segurança.

# A diferença entre dizer o que houve e dizer o que fazer

| SITUAÇÃO          | INÚTIL                                 | ÚTIL                                                                 |
|-------------------|----------------------------------------|----------------------------------------------------------------------|
| Erro de validação | <code>"Dados inválidos"</code>         | <code>"O campo 'birthDate' deve estar no formato AAAA-MM-DD."</code> |
| Recurso ausente   | <code>"Erro ao buscar"</code>          | <code>"Paciente não encontrado."</code>                              |
| Conflito          | <code>"Não foi possível salvar"</code> | <code>"Já existe um paciente com este CNS."</code>                   |

O teste: a pessoa que recebeu a mensagem sabe o que fazer agora?

Mas cuidado com o outro extremo — nunca devolva o erro bruto do banco. `"SQLITE_CONSTRAINT: UNIQUE failed: patients.national_id"` entrega ao atacante o nome da sua tabela e da sua coluna.

Sua rota **POST /api/patients** responde ``200`` com ``{ "error": "name obrigatório" }``. O que há de errado?

- A** Nada. O erro está descrito no corpo da resposta.
- B** Deveria ser 500: o servidor não completou a operação.
- C** Deveria ser 4xx: o status é parte do contrato, e programas leem o status, não o corpo.
- D** Deveria ser 204, porque nenhum recurso foi criado.

**Vote → discuta com a dupla por 2 minutos → vote de novo.**

# A rota POST

TODO 2 em `src/server.ts`. Teste com o `requests.http` — os casos de falha já estão escritos.

1

## Crie a rota e imprima o corpo

Só ``console.log(request.body)`` primeiro. Confirme que chega.

2

## Valide antes de tudo

`name`, `birthDate` (AAAA-MM-DD) e `nationalId`. Devolva 400 na primeira falha.

3

## Responda 201 com o recurso

Acrescente ao array em memória e devolva o objeto criado, com `id`.

4

## Rode os casos 5 a 9

Todos. Inclusive o 9, que quebra de propósito. Entenda por que quebrou.

### CRITÉRIO DE PRONTO

#### Como saber que acabou

- Caso 5 → 201 com o paciente e um `id`
- Casos 6, 7 e 8 → 400 com mensagem diferente em cada
- Caso 9 → você sabe explicar por que falhou
- Nenhum 500 em nenhum dos casos

### NÍVEL 3

#### Para quem terminou

- Rejeite data de nascimento no futuro
- Devolva TODOS os erros de uma vez, num array

BLOCO 03

# Persistência sem ORM

Você já sabe SQL. Hoje ele vai para dentro da aplicação — do jeito seguro.



# Por que não usamos ORM nesta semana

O Plano de Ensino prevê ORM. Ele chega — na Semana 4. Hoje ele atrapalharia.

## O QUE UM ORM FAZ

### Esconde o SQL

`Patient.findAll()` gera o SELECT para você. Em time grande e projeto longo, isso economiza muito trabalho e evita erro bobo.

É o padrão de mercado — e vocês vão usar.

## POR QUE NÃO AGORA

### Você não sabe o que esconder

Esconder é útil quando você conhece o que está atrás. Antes disso, é só mistério.

Vocês fizeram duas disciplinas de Banco de Dados. Seria estranho passar por cima disso.

## O QUE USAMOS

### **better-sqlite3**

Você escreve o SQL. A biblioteca só executa.

API síncrona: `.all()` devolve o array direto, sem Promise e sem callback. Menos ruído entre você e o conceito.

Quando o ORM chegar na Semana 4, vocês vão saber exatamente qual SQL ele está gerando — e quando ele está gerando um SQL ruim.

# Quatro métodos. É a API inteira que você precisa hoje.

SRC/DATABASE.TS E SRC/SERVER.TS

```
import Database from 'better-sqlite3';
export const db = new Database('database/prontuario.db');
db.pragma('foreign_keys = ON'); // SQLite exige ligar

// .all() -> array de linhas
const rows = db
  .prepare('SELECT id, name FROM patients ORDER BY name')
  .all();

// .get() -> UMA linha, ou undefined
const row = db
  .prepare('SELECT * FROM patients WHERE id = ?')
  .get(42);

// .run() -> para INSERT, UPDATE, DELETE
const result = db
  .prepare('INSERT INTO patients (name) VALUES (?)')
  .run('Joana');
result.lastInsertRowid; // o id que o banco gerou
```

`.PREPARE()`

## Compila uma vez

O banco analisa o comando e guarda o plano. Rodar de novo com outro valor é barato.

`.GET()` DEVOLVE `UNDEFINED`

Não lança erro quando não acha. É você quem decide que isso vira um 404.

SÍNCRONO

Sem `.await`. Só funciona bem porque SQLite é um arquivo local — não faça isso com banco em rede.

# O banco fala `snake\_case`. A API fala `camelCase`.

## A TRADUÇÃO EXPLÍCITA

```
type PatientRow = {           // como vem do banco
  id: number;
  name: string;
  birth_date: string;
  national_id: string;
  active: number;             // SQLite nao tem boolean
};

function toPatientJson(row: PatientRow) {
  return {
    id: row.id,
    name: row.name,
    birthDate: row.birth_date,
    nationalId: row.national_id,
    active: row.active === 1,   // 0/1 vira true/false
  };
}
```

## POR QUE NÃO DEVOLVER A LINHA CRUA

### O banco é seu, não do cliente

Se a API expõe o formato da tabela, renomear uma coluna quebra todo mundo que integrou com você.

## BÔNUS DE SEGURANÇA

### Você escolhe o que sai

Nesta função é impossível vazar por acidente uma coluna nova que alguém adicionou na tabela.

## SPOILER DA SEMANA 3

Esse tradutor tem nome: mapeamento entre modelo de dados e modelo de resposta.

# É a diferença entre dado e código

## CONCATENANDO (NUNCA FAÇA)

```
const sql =
  `SELECT * FROM patients WHERE name = '${nome}'`;
db.prepare(sql).all();

// se nome = Maria
SELECT * FROM patients WHERE name = 'Maria'

// se nome = '; DROP TABLE patients; --
SELECT * FROM patients WHERE name = ';
DROP TABLE patients; --'
```

## PARAMETRIZADO (SEMPRE)

```
db.prepare('SELECT * FROM patients WHERE name = ?')
  .all(nome);

// se nome = '; DROP TABLE patients; --
// o banco procura um paciente chamado
// literalmente '; DROP TABLE patients; --
// e nao acha nenhum.

// O comando ja foi compilado ANTES do valor chegar.
// Dado nunca vira comando. Nunca.
```

Injeção de SQL está na OWASP Top 10 desde a primeira edição, em 2003. Vinte e três anos depois, continua entre as falhas mais exploradas do mundo. Não porque seja difícil de evitar — mas porque alguém, em algum lugar, achou mais rápido concatenar.

``db.prepare("SELECT * FROM patients WHERE name = ?").all(nome)`` é mais seguro porque:

- A** É mais rápido, e ataques dependem de lentidão.
- B** O valor nunca é interpretado como SQL — o driver o trata sempre como dado.
- C** O SQLite valida automaticamente o conteúdo de qualquer parâmetro.
- D** O TypeScript impede que uma string maliciosa chegue até ali.

Se você marcou D, cuidado: TypeScript some em tempo de execução. Ele não protege nada em produção.

# Do array em memória ao banco de verdade

TODO 3. E, no fim, a troca de uma linha que eu prometi no começo da aula.

1

**`npm run db:reset`**

Abra o arquivo .db com o SQLite Viewer. Veja as 8 linhas com seus olhos.

2

**Troque o array pelo SELECT**

Em GET /api/patients. Não esqueça do `toPatientJson`.

3

**INSERT no POST + `GET /:id`**

404 quando não existir. Todo SQL parametrizado.

4

**A troca de uma linha**

Em api.js: `./mock/patients.json` vira `/api/patients`. Só isso. Recarregue.

## CRITÉRIO DE PRONTO

### O teste final

- Cadastre pelo requests.http e recarregue a página
- O paciente novo aparece na lista do navegador
- Reinicie o servidor: ele continua lá
- Feche tudo, abra amanhã: continua lá

## SE TRAVAR

### Perguntas que destravam

- `no such table` → rodou o db:reset?
- Campos `undefined` no front → passou pelo tradutor?
- `active` aparece como 0/1 → faltou `=== 1`

O MOMENTO DA SEMANA

# Uma linha

ANTES

```
const PATIENTS_URL = './mock/patients.json';
```

DEPOIS

```
const PATIENTS_URL = '/api/patients';
```

`state.js`

não mudou

`render.js`

não mudou

`app.js`

não mudou

o CSS

não mudou

Isso tem nome: acoplamento baixo. E não foi sorte — foi a consequência direta de uma decisão que vocês tomaram na terça, quando concentraram todo `fetch` em um arquivo só.

Essa é a diferença entre código que funciona e código que aguenta mudar.

# O caso do CNS repetido

## SEM TRATAMENTO: O INSERT ESTOURA

```
SqliteError: UNIQUE constraint failed:
  patients.national_id

// O Express devolve 500 + stack trace na resposta.
// Duas coisas erradas de uma vez:
// 1. o servidor esta otimo, quem errou foi o cliente
// 2. voce acabou de entregar ao atacante o nome
// da sua tabela e da sua coluna
```

## COM TRATAMENTO

```
const duplicate = db
  .prepare('SELECT id FROM patients
           WHERE national_id = ?')
  .get(nationalId.trim());

if (duplicate) {
  response.status(409).json({
    error: 'Ja existe um paciente com este CNS.'
  });
  return;
}
```

## HONESTIDADE TÉCNICA

### Esta solução tem uma falha — e você precisa saber qual

Entre o SELECT e o INSERT existe uma janela. Se duas requisições chegarem no mesmo instante, as duas podem passar pela verificação.

A solução robusta é capturar o erro do banco (que é atômico) em vez de perguntar antes. Vamos fazer isso quando falarmos de transações. Por hoje, o importante é você saber que a janela existe.

# Os modos de falha mudam quando o assunto é servidor

Mesmas três zonas, mesmo protocolo V.E.R.M. Só a lista de armadilhas é outra.

! Vai sugerir Prisma, TypeORM ou Sequelize quando você pediu SQL explícito

! Vai importar ``body-parser`` — desnecessário desde o Express 4.16

! Vai misturar ``require`` e ``import`` no mesmo arquivo

! Vai concatenar string no SQL se você não escrever “parametrizado” no prompt

! Vai sugerir ``sqlite3`` (assíncrono, com callbacks) em vez de ``better-sqlite3``

! Vai adicionar ``cors`` e ``dotenv`` que não pedimos e não precisamos

! Vai inventar métodos que não existem em ``better-sqlite3``

! Vai propor separação em camadas que ainda não temos

**O prompt que resolve quase tudo: “Node 22, Express 5, TypeScript, better-sqlite3. SEM ORM, SEM bibliotecas além dessas, SEM separação em camadas, SQL parametrizado. Devolva só o trecho da rota.”**

# Registrando o atendimento

O Mini-Prontuário sabe cadastrar paciente. Falta a razão de existir de um prontuário: o registro do atendimento.

## NÍVEL 1 · ESSENCIAL

### A API de Encounter

- Tabela `encounters` com chave estrangeira
- GET /api/patients/:id/encounters
- POST na mesma rota, com validação
- Todo SQL parametrizado
- requests.http com sucesso e falha

## NÍVEL 2 · PROFICIENTE

### A tela de detalhe

- Painel do paciente com seus atendimentos
- Formulário com erro vindo do backend
- 404 tratado no frontend
- Componente `.encounter-item` em BEM, testado em 375px

## NÍVEL 3 · AVANÇADO

### Portfólio

- Estados de carregamento e erro na interface
- Ordenação justificada no README: SQL ou JS?
- Contador de atendimentos derivado do estado
- Diagrama do fluxo completo

Entrega: repositório com commits das DUAS pessoas · README · IA.md · autoavaliação de 5 linhas. Quem ficar no Nível 1 pode reentregar no Nível 2 até o fim da Semana 3, sem penalidade.

# Uma pergunta só, e ela vale para o seu portfólio

Descreva, em três linhas e para alguém do 1º módulo, o que acontece entre clicar em “Cadastrar” e o paciente aparecer na lista.

1

## Cite os quatro

O que sai do navegador, o que o Express faz com isso, o que vai para o banco, o que volta.

2

## Sem jargão vazio

“Faz uma requisição” não explica nada. Diga o método, o caminho e o que vai no corpo.

3

## Guarde este texto

Ele é o rascunho do resultado de aprendizagem RA1 e vai para o seu portfólio da disciplina.

# Cinco frases

- 1 Toda requisição tem método, caminho, cabeçalhos e corpo.
- 2 O status é o contrato. 4xx é culpa do pedido, 5xx é culpa minha.
- 3 Sem `express.json()`, `req.body` é `undefined`.
- 4 O `?` faz o dado nunca virar comando.
- 5 Trocar de fonte de dados custou uma linha — porque o `fetch` mora num lugar só.

*Semana 2: API REST de verdade, contratos e o que “Restful” realmente significa.*